CS303 Project 2 Report

Subtask 1: Image Classification

1. Introduction

The first subtask is a supervised image-vector classification problem. Each image is already represented as a 256-dimensional feature vector, and the goal is to infer one of ten class labels for unseen test samples.

The provided baseline is Softmax Regression. A linear classifier is efficient, but its decision boundary is limited to linear class separation in the feature space. My method uses an ensemble of multilayer perceptrons (MLPs) to model nonlinear feature interactions while preserving a simple submitted inference interface.

2. Methodology

Let the normalized input be

$$\tilde{x} = \frac{x-\mu}{\sigma}.$$

For each MLP model, hidden layers use the ReLU activation

$$h^{(l)} = \max(0, h^{(l-1)}W^{(l)} + b^{(l)}),$$

and the final output is transformed by softmax:

$$p_c(x)=\frac{\exp(o_c)}{\sum_j \exp(o_j)}.$$

The ensemble prediction averages posterior probabilities from eight independently initialized MLPs:

$$\bar{p}(x)=\frac{1}{M}\sum_{m=1}^{M}p_m(x),\qquad
\hat{y}=\arg\max_c \bar{p}_c(x).$$

Pseudo-code:

```
fit scaler on training features
for each MLP configuration and random seed:
    train MLP with early stopping
    export weights and biases
for inference:
    normalize X
    average all MLP softmax probabilities
    return argmax labels
```

The submitted `classifier.py` does not unpickle sklearn models. The training script exports all learned weights to `classification_mlp_ensemble.npz`, and inference is implemented with NumPy matrix multiplication. This avoids compatibility risk across sklearn versions.

The time complexity for one inference batch is

$$O\left(M\sum_l n d_l d_{l+1}\right),$$

where $M=8$, n is batch size, and d_l are layer widths. The decisive factor is the ensemble's ability to reduce variance across random MLP initializations.

3. Experiments

The metric is classification accuracy:

$$\text{Accuracy}=\frac{\#\text{correct predictions}}{\#\text{samples}}.$$

I used a stratified 80/20 validation split with random seed 123.

Model	Hidden Layers	Validation Accuracy
MLP seed 123	512	0.5495
MLP seed 456	512	0.5445
MLP seed 789	512	0.5436
MLP seed 2026	512	0.5434
MLP seed 123	256	0.5341
MLP seed 456	256	0.5384
MLP seed 123	512, 256	0.5337
MLP seed 456	512, 256	0.5261
Probability ensemble	all 8 models	**0.5877**

For comparison, local linear baselines were approximately 0.4915 for Ridge Classifier and 0.5062 for Logistic Regression. The ensemble therefore improves substantially over linear softmax-style classification.

4. Further Thoughts

With more time, I would tune the MLP widths and regularization with cross-validation, then use calibration or weighted averaging so stronger models contribute more to the final probability ensemble.

Subtask 2: Image Retrieval

1. Introduction

The second subtask is unsupervised image retrieval. Given a query feature vector, the model must return five similar repository images. The baseline uses nearest-neighbor search with raw Euclidean distance.

The main challenge is that no retrieval labels are provided. Therefore, the practical objective is to choose a similarity metric that is more robust than raw Euclidean distance while preserving the baseline output convention.

2. Methodology

The selected method is standardized Euclidean nearest-neighbor search. Each feature dimension is normalized using repository statistics:

```
z_i = \frac{x_i-\mu_i}{\sigma_i}.
```

Then query-to-repository distance is

```
d(q,r)=\sum_{i=1}^{256}(z_{q,i}-z_{r,i})^2.
```

The five repository rows with the smallest standardized distance are returned. Raw Euclidean distance is used only as a deterministic tie-breaker. The output remains repository row indices, matching the provided baseline implementation.

Pseudo-code:

```
compute repository mean and standard deviation
standardize repository features
for each query:
    standardize query with repository statistics
    compute squared Euclidean distances
    return the 5 smallest-distance repository row indices
```

The complexity is

$$O(nqd),$$

where n is repository size, q is the number of queries, and $d=256$.

3. Experiments

Because hidden query relevance labels are unavailable, I used a repository self-query proxy. For each repository vector, I excluded itself and measured how often the top-5 neighbors shared the class label associated with the repository image id. This proxy is weak but useful for comparing metric choices.

Similarity Metric	Proxy Same-Class Top-5
Raw Euclidean baseline	0.09860
Standardized Euclidean	**0.10016**
Raw cosine	0.09860
Standardized cosine	0.09972

The improvement is small, which is expected because the proxy labels are not the official retrieval relevance labels. Still, standardizing feature dimensions removes scale imbalance and gave the best observable result.

4. Further Thoughts

If official relevance examples were available, I would learn a metric directly, for example with contrastive learning or a supervised Mahalanobis distance. Without relevance labels, standardized nearest-neighbor search is a conservative improvement over the raw-distance baseline.

Subtask 3: Feature Selection

1. Introduction

The third subtask asks for a binary mask selecting no more than 30 out of 256 features. The selected mask is evaluated by a fixed image recognition model, so the task is to preserve the most useful coordinates for that fixed classifier.

The baseline randomly selects 30 dimensions. My method uses validation labels and the fixed model weights to search for a feature subset with much higher validation accuracy.

2. Methodology

The fixed classifier applies a linear score after masking:

$$o = W(x\odot m)+b,$$

where $m\in\{0,1\}^{256}$ and $\sum_i m_i=30$. The objective is

$$\max_m \frac{1}{N}\sum_{j=1}^{N}\mathbf{1}\left[
\arg\max_c o_{j,c}=y_j
\right].$$

The final mask was produced by beam forward selection followed by a one-swap local check:

```
start with empty mask
repeat until 30 features are selected:
    expand the best beam states by adding one unused feature
    keep the best beam states by validation accuracy
try all one-feature swaps
keep a swap only if validation accuracy improves
```

This directly optimizes the same accuracy used by the evaluation notebook. The selected feature indices are:

```
0, 2, 3, 4, 5, 6, 7, 9, 11, 12,
13, 14, 15, 16, 17, 19, 20, 24, 26, 28,
29, 42, 46, 47, 59, 62, 71, 120, 130, 221
```

3. Experiments

The metric is the validation accuracy of the fixed recognition model after applying the mask:

```
\text{Accuracy}=\frac{1}{N}\sum_j \mathbf{1}[\hat{y}_j=y_j].
```

Method	Selected Features	Validation Accuracy
Random seed 42 baseline	30	0.1649
Single-feature ranking top 30	30	0.4296
Greedy forward selection	30	0.4556
Backward elimination	30	0.4594
Beam forward selection + swap check	30	**0.4646**

The selected mask is binary with shape \(1\times256\), and it selects exactly 30 features.

4. Further Thoughts

The feature search is non-convex because feature interactions can change the fixed model's argmax decisions. Wider beam search or stochastic local search could potentially find a slightly better subset, but the current mask already improves sharply over random selection.

Final Submission Notes

The submission directory includes:

- `task1/classifier.py` and `task1/classification_mlp_ensemble.npz`
- `task2/retrieval.py`
- `task3/selector.py` and `task3/mask_code.pkl`
- this report in Markdown and PDF form

The original provided files under `original_files/` were not modified.